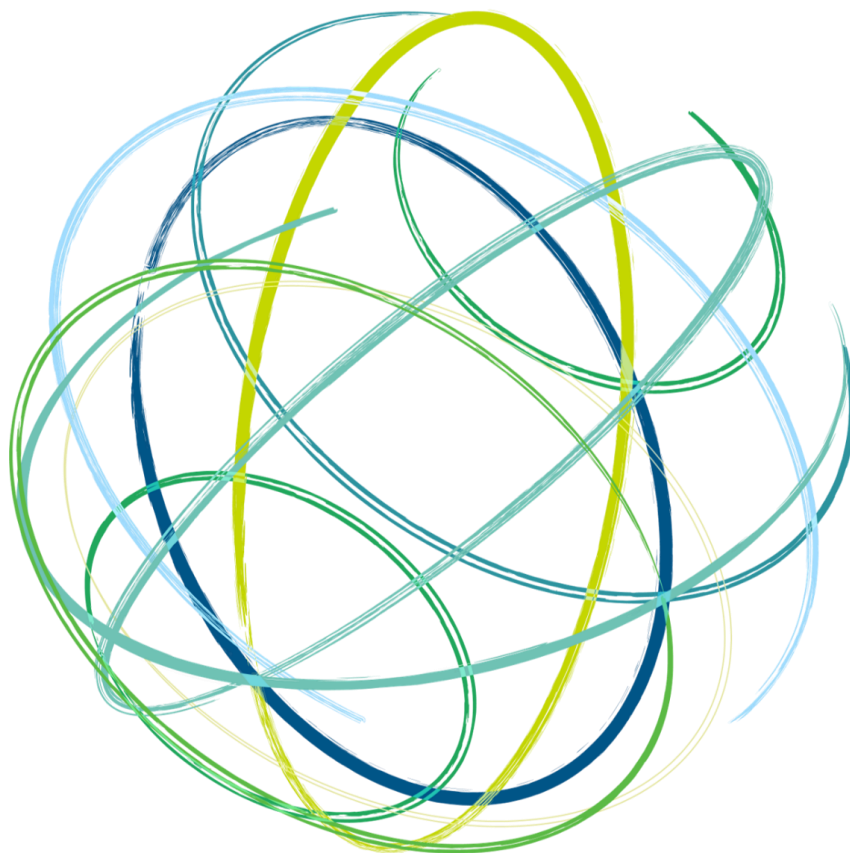




ANTHROPIC TRAINING PROGRAM • HACKATHON

Challenge Brief

Claude Code Live

Two scenarios. 2 hours of build. One submission. Read this before you do anything else.

Document ID	3.4 —Participant-facing—for 18 June 2026
Version	v1.5
Date	2026-08-25 (v1.5: submission mechanism, reflection, and build-window corrections)
Owner	Pradeep Joseph —Learning Design Services (LDS)
Audience	Deloitte Claude Trainer cohort (Train-the-Trainer)
Classification	Internal—LDS draft

What you are doing here

You have 2 hours. In that window you will pick one of two scenarios, build something against it with Claude Code, and submit. The team that produced the training you completed is now asking you to show what stuck. This brief tells you what to build, how to submit it, and what we will be looking for.

The next 2 hours

When	What's happening	Where
0–15 min	Welcome + this brief in your hands	Hackathon Pack
15–120 min	Build (1 hour, 45 minutes)	Your team, your tools, your Claude Code env
Before 120 min	Add Submission to shared folder	Shared folder or channel (set by your facilitators)
<i>Next Module:</i> 30 min	Module 6: Advanced Overview – API, SDK, and Managed Agents	
Final Module: 20 min	Winners announced + Wrap Up	Find out who won. See you there.
0–15 min	Welcome + this brief in your hands	Hackathon Pack

Pick one

Two scenarios. Pick the one closer to your day job. There is no extra credit for picking the harder one. Pick where you can show your best work in 2 hours.

Scenario A • Code Modernization

You have been handed a 150-line Python script—`invoice_processor.py`—that currently reads a CSV of expense transactions, applies a set of categorization and approval rules, and outputs a summary report. It works. It is also a mess. Variable names are unhelpful, logic is duplicated, exception handling is indiscriminate, and there is at least one bug that has been there for so long nobody noticed. Your job is to modernize it using Claude Code.

What you start with

File	What it is
<code>invoice_processor.py</code>	The legacy script. ~150 lines. In your Hackathon pack.

transactions.csv	Sample input data the script reads from. In your Hackathon pack.
test_invoice_processor.py	A small regression suite. Run it as you go: <code>pytest test_invoice_processor.py -v</code> . It checks your refactor still does what the original did. One test only passes once you fix the bug.

What you must produce

Artifact	What good looks like
A refactored Python file	Behaves the same as the original on the sample data (the regression tests are how you prove it). Readable. Idiomatic. Functions where there were none. Names that mean something. Exposes a <code>summarize(path)</code> function—see the test file's header.
A CLAUDE.md	Your direction to Claude Code on this project. Use the scaffold in the Hackathon pack. Filled in, not blank—including Section 7, the direction log.

What we will be looking for

- The refactored code is something you would be willing to push to a client repo.
- The regression tests pass—that is how you prove the original behavior is preserved.
- The last test only goes green once you find and fix the bug. (There is at least one. Not telling you where.)
- Your CLAUDE.md tells Claude what good looks like, what is in scope, and what is not.
- You used plan mode at least once before letting Claude write code.

What to avoid

- Wholesale rewrite that loses the original behavior without flagging the change.
- A CLAUDE.md that just describes Python in general terms—it should describe THIS project.
- Optimization work the brief did not ask for (performance, async). Refactor first; anything else only if you have time.
- Letting Claude do everything without pushing back. The point is to direct, not to ride.

Scenario B · Data Analytics

You have been handed a CSV—engagement_timesheets.csv—covering three months of practitioner timesheet entries from a fictional Deloitte engagement. The data is messy: inconsistent client codes, free-text task descriptions, missing entries, a handful of obvious anomalies. Your job is to surface one insight that the engagement lead would want to know about, produce a chart that makes it visible, and write a short narrative explaining what you found.

What you start with

File	What it is
engagement_timesheets.csv	~150 rows, ~8 columns. Practitioner ID, date, client code, project code, task description (free text), hours, billable flag, status. In your Hackathon pack.

What you must produce

Artifact	What good looks like
Your analysis code	Whatever you used to interrogate the data—Python, a notebook, a script, Claude artifacts. Reproducible by a colleague.
A chart	One chart that shows the insight clearly. PNG, screenshot, or the code that generates it. Not a wall of charts—one.
A CLAUDE.md	Your direction to Claude on this analysis. Use the scaffold in the Hackathon pack. Filled in, not blank.

What we will be looking for

- The insight is something a non-data-engineer can grasp in two sentences.
- The chart shows the insight, not the entire dataset.
- Your CLAUDE.md sets up the analytical frame—what you were looking for and why.
- You did not chase every anomaly—you picked the one that mattered.
- Your narrative is honest about confidence (for example, 'this looks like X, but a clean month of data would confirm it').

What to avoid

- Five insights and three charts. We are asking for one.
- A dashboard. We did not ask for a dashboard.
- A CLAUDE.md that describes pandas. It should describe THIS analysis.

- Cleaning the data forever and never producing the insight. Cleaning is means, not end.

What's in your Hackathon pack

Item	What it's for
This Brief	What you are reading. The thing you came here for.
CLAUDE.md scaffold (.md)	Seven section prompts. Open in your editor. Fill in the prompts. Use it from the start.
For Scenario A · invoice_processor.py	The legacy script. Drop it into your project folder.
For Scenario A · transactions.csv	Sample data the script reads. Drop it next to the script.
For Scenario A · test_invoice_processor.py	The regression suite. Run it as you refactor: <code>pytest test_invoice_processor.py -v</code> .
For Scenario B · engagement_timesheets.csv	The dataset you will be analyzing. Drop it into your project folder.

How to start

#	Step	Why
1	Open Claude Code in your own environment	Same setup as Code Live. If something is broken on your machine, raise it in the help channel your facilitators set up—facilitators are watching the channel.
2	Author your CLAUDE.md before you write any code	Use the scaffold. Fill in what the project is, what good looks like, what is out of scope, conventions you will follow, the input files, the tone the artifact should land in. Five focused minutes here saves you twenty later.
3	Use plan mode first	Get Claude to lay out the steps. Read it. Push back where it is wrong. Then execute. Do not let it skip plan mode—that is where the discrimination happens.
4	Build and remember to polish.	Stop adding scope 10 minutes before submission. The last 10 minutes are for tightening the artifact and the CLAUDE.md, not for one more feature.
5	Submit	Submit to the shared folder or channel your facilitators set up.

How you'll be judged

How judging works: Claude scores every submission — your artifact and your CLAUDE.md — using a judging prompt designed, tested, and validated by Deloitte Technology Academy. Facilitators review Claude's findings

before any winners are confirmed. Build the best, most honest work you can; don't write for a checklist. Winners are announced at the close.

Submission

Submissions will be uploaded to a shared folder or channel your facilitators designate.

Help, rules, etiquette

- Cameras on for kick-off and close. Off during build if your team wants—it is your call.
- Stuck? Ask Claude. If still stuck, ask the channel. Do not silently lose 20 minutes.
- Submission: one per team. The team's nominated submitter sends it. Last submission wins.
- Be generous with each other. The judging is about your team's work, not your team's hierarchy.

Prepared by Learning Design Services.